

구조체 · 비트 연산 · 함수 포인터

다음 주 ESP32 레지스터를 만지는 문법이 오늘 4.5교시. 오늘 함정은 죽지 않고 조용히 틀린다



오늘의 8교시

오전은 데이터 모양(구조체·비트), 오후는 코드 모양(매크로·함수 포인터·상태 머신)

1

구조체 기초

. -> 통째 복사, const T *

2

패딩·union·enum

정렬 규칙, pack, ST_COUNT

3

비트 연산 기초

& | ^ ~ << >>, 4 관용구

4

비트 연산 실전

GPIO 매크로, 플래그, XOR

5

매크로·volatile

괄호 셋, -DDEBUG, 메모리 맵

6

함수 포인터

선언 읽기, 테이블, qsort

7

콜백·상태 머신

등록 패턴, TABLE[st](t)

8

종합 실습

온도 조절기 컨트롤러

구조체 — 묶고, 복사하고, 가리키기

1교시

```
typedef struct { double t; int h; int d; } Reading;
```

```
Reading r = {25.3, 60, 120};
```

```
Reading s = {.d = 150, .t = 36.9}; // 빠진 h 는 0
```

```
Reading c = r; // 16바이트 통째 복사
```

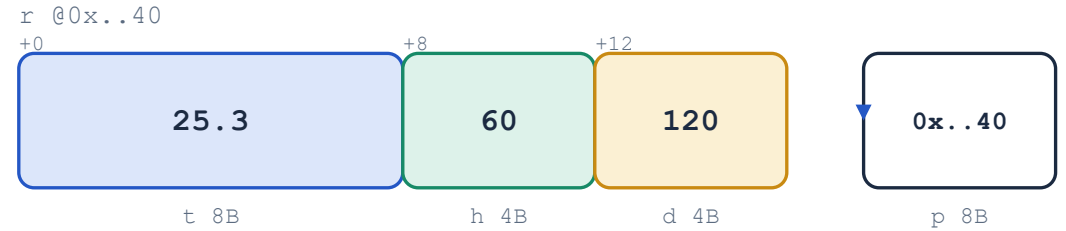
```
c.t = 99.0; // r.t 는 25.3 그대로
```

```
Reading *p = &r;
```

```
p->h == (*p).h // 60
```

```
void bump(Reading r) { r.t += 1; } // 사본
```

```
void bump(Reading *r) { r->t += 1; } // 원본
```



- 필드 이름은 사라지고 오프셋만 남는다. `p->h` 는 "p 주소 +8 을 int 로 읽어라"
- 구조체 대입 = `memcpy`. 배열은 = 로 복사 안 되지만 구조체에 쓰면 된다 (안의 배열까지)
- `==` 비교는 없다. 필드별 비교 함수. `memcmp` 는 패딩 때문에 못 믿는다
- 함수 인자는 `const T *` 가 기본 — 어제의 "원본을 바꾸려면 주소"

12 가 아니라 16 — 정렬과 패딩

2교시

`struct { char id; int v; char f; } → sizeof 12`



`struct { int v; char id; char f; } → sizeof 8`



- 규칙 ① 각 필드는 자기 크기의 배수 주소에서 시작 (int 는 4, double 은 8)
- 규칙 ② 전체 크기는 가장 큰 필드의 배수 (배열로 늘어놔도 ①이 유지되게)
→ 큰 것부터 놓으면 패딩이 사라진다

#pragma pack(1): 패딩 0 이지만 정렬 안 된 접근 → x86 느려짐, 일부 MCU 죽음. 프레임 캐스팅 (Frame *)buf 의 유혹 — 안전한 답은 바이트 조립(4교시)

union — 같은 자리를 세 이름으로

```
typedef union { uint32_t w; uint8_t b[4]; float f; } Word;
Word x; x.w = 0x11223344;
x.b[0..3] // 44 33 22 11 (LE)
x.f = 1.0f; x.w // 0x3F800000 (IEEE 754)
sizeof(Word) // 4 (가장 큰 멤버)
```

enum — 상태에 이름을, 그리고 개수를

```
typedef enum { ST_IDLE, ST_HEAT, ST_COOL,
              ST_ALERT, ST_COUNT } State; // 0..3,
4
const char *NAME[ST_COUNT] = {"IDLE",...};
for (i = 0; i < ST_COUNT; i++) ...
State st = 99; // 컴파일됨! enum 은 int - default 로
방어
```

비트 연산 — 여섯 연산자, 네 관용구

3교시

연산	규칙	용도	예
$a \& m$	둘 다 1	마스크로 남기기	$a \& 0x0F \rightarrow$ 하위 4비트
$a m$	하나라도 1	켜기	$a (1u \ll 3)$
$a \wedge m$	다르면 1	토글·체크섬	$a \wedge 0xFF \rightarrow$ 반전, $a \wedge b \wedge b == a$
$\sim a$	반전	AND NOT 마스크	$\sim (1u \ll 3)$
$a \ll n$	n칸 왼쪽	$\times 2^n$, 자리 올리기	$\text{buf}[1] \ll 8$
$a \gg n$	n칸 오른쪽	$\div 2^n$, 자리 내리기	$(\text{reg} \gg 5) \& 1$

```
reg |= (1u << n); // 켜기
reg &= ~(1u << n); // 끄기
reg ^= (1u << n); // 토글
(reg >> n) & 1 // 검사
```

// 필드 쓰기: 지우고 넣는다

```
reg = (reg & ~(0xFFu << 8)) | (0xABu << 8);
// 0x12345678 → 0x1234AB78
```

△ `uint8_t a; a << 1` 은 356 — int 로 정수 승격(Day 1). 8비트로 자르려면 `(uint8_t)(a << 1)`. `~a` 도 마찬가지.
△ `1 << 31` 은 signed 오버플로. 레지스터는 항상 1u. 음수 `>>` 는 구현 정의 — 비트 연산은 unsigned 로만.

가상 GPIO — 비트가 편이다

4교시

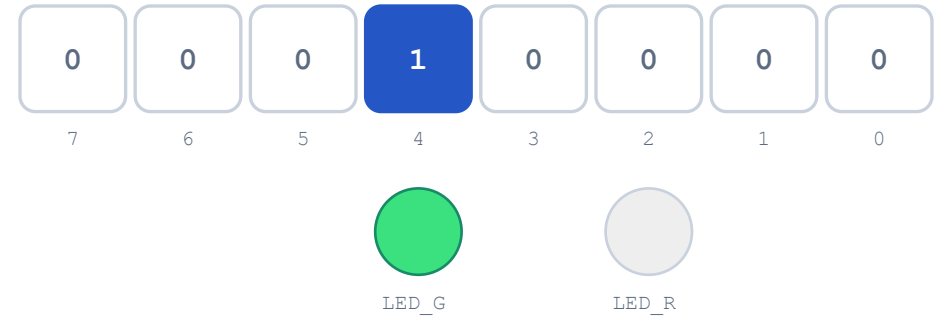
다음 주엔 GPIO_OUT 한 줄만 진짜 주소로 바뀐다. 매크로는 그대로

```
extern uint32_t GPIO_OUT;    // MCU: (*(volatile uint32_t
*)0x3FF44004)
#define PIN_LED_R 2
#define PIN_LED_G 4

#define GPIO_SET(pin)      (GPIO_OUT |=  (1u << (pin)))
#define GPIO_CLR(pin)      (GPIO_OUT &= ~(1u << (pin)))
#define GPIO_TOGGLE(pin)   (GPIO_OUT ^=  (1u << (pin)))
#define GPIO_READ(pin)     ((GPIO_IN >> (pin)) & 1u)

GPIO_SET(PIN_LED_R);    // 0x00000004
GPIO_SET(PIN_LED_G);    // 0x00000014
GPIO_CLR(PIN_LED_R);    // 0x00000010
```

GPIO_OUT 하위 8비트 = 0x10



```
// D=120 = 0b0111_1000 를 플래그로
#define FLAG_ALARM  (1u << 5)
#define LEVEL_MASK  0x0Fu
!!(d & FLAG_ALARM)  // 1 (32 가 아니라)
d & LEVEL_MASK      // 8

// XOR 체크섬 $...*hh (NMEA 방식)
for (; *s && *s != '*'; s++) cs ^= *s;    // 2C
```

⚠ 매크로는 함수가 아니라 텍스트 치환이다

5교시

```
#define SQ_BAD(x)  x * x
#define SQ(x)      ((x) * (x))
#define MAX(a, b)  ((a) > (b) ? (a) : (b))

SQ_BAD(1 + 2)      // 1 + 2 * 1 + 2 = 5   (9 아님)
SQ(1 + 2)          // ((1+2) * (1+2)) = 9
100 / SQ(5)        // 전체 괄호 없으면 100/5*5 = 100
int i = 3;
MAX(i++, 2)        // → 4, i = 5 !!  두 번 평가
```

규칙 셋: 인자마다 괄호 · 전체에 괄호 · 부수효과 있는 인자 금지.
타입 검사도 없다 → 요즘은 static inline 함수 (bits.h 의 print_bin8).

```
#ifdef DEBUG
#   define LOG(fmt, ...) \
        fprintf(stderr, "[%s:%d] " fmt "\n", \
                __FILE__, __LINE__, ##__VA_ARGS__)
#else
#   define LOG(fmt, ...) ((void)0)    // 코드가 사라진다
#endif

$ gcc dbg.c                # done
$ gcc -DDEBUG dbg.c        # [dbg.c:11] temp=37 ...
```

-DDEBUG = 소스 첫 줄 #define DEBUG. 릴리스에서 LOG 는 플래시·속도 비용 0. #if defined(ESP32) 플랫폼 분기도 같은 문법.

volatile — "매번 진짜로 읽어라" · 메모리 맵 I/O

5교시

```
uint32_t      plain_flag = 0;
volatile uint32_t hw_flag  = 0;

while (plain_flag == 0) {}
// -O2: "아무도 안 바꾸니 영원히 참" → 무한 루프로 최적화

while (hw_flag == 0) {}
// volatile: 매번 메모리에서 다시 읽음
```

volatile 이 필요한 곳

- 하드웨어 레지스터
- 인터럽트 핸들러가 바꾸는 변수
- 다른 스레드의 플래그 (동기화는 별도)
- 일반 루프 카운터 — 최적화만 막힘

주소를 레지스터로

```
// 실제 ESP32 (다음 주)
#define GPIO_OUT_REG (*(volatile uint32_t *)0x3FF44004)

GPIO_OUT_REG |= (1u << 2); // 그 주소에 쓰기

// 읽는 법: 주소 → volatile uint32_t 포인터로 캐스팅 → 역참조
// 그러면 변수처럼 |= &= 를 쓸 수 있다
```

4교시 gpio.h 의 extern uint32_t GPIO_OUT 한 줄만 이걸로 바꾸면 실제 하드웨어 코드. 하드웨어 의존을 한 파일 한 줄에 가두는 것이 이식성의 구조.

함수도 주소가 있다 — 함수 포인터

6교시

```
int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

int (*op)(int, int); // "op 는 (int,int)→int 함수의 포인터"
op = add;           // 함수 이름 = 주소
op(2, 3);           // 5

typedef int (*BinOp)(int, int);
static const BinOp OPS[] = { add, sub, mul };
OPS[2](7, 3);       // 21 ← switch 없는 분기
```

OPS[0] → add

int add(){ a+b }

OPS[1] → sub

int sub(){ a-b }

OPS[2] → mul

int mul(){ a*b }

인덱스가 함수를 고른다. 연산 추가 = 배열 한 항목

Day 1 6교시 switch 계산기 → 표 한 줄. 이것이 7교시 상태 머신 테이블의 원리.

괄호를 빼면 int *op(int, int) — "int 포인터를 돌려주는 함수" 선언. 전혀 다른 뜻. 복잡하면 typedef.

qsort — 어제의 버블 정렬을 라이브러리로

6교시

```
int cmp_temp(const void *a, const void *b) {
    const Reading *x = a, *y = b;          // void* → 원래 타입
    return (x->t > y->t) - (x->t < y->t); // -1 / 0 / 1
}

int cmp_hum_desc(const void *a, const void *b) {
    return ((const Reading *)b)->h - ((const Reading *)a)->h; // b-a:
내림차순
}

qsort(log, n, sizeof log[0], cmp_temp);
//   배열  개수  요소 크기   비교 함수
// 24.8 26.0 37.2
```

⚠ return `*(int*)a - *(int*)b`; 는 `INT_MAX - (-1)` 에서 오버플로 → 잘못된 순서. 값이 작을 때만 우연히 맞는다.

배열, 개수, 요소 크기

Day 2 "길이를 같이 넘겨라"의 완성형 (감쇠)

const void *

qsort 는 타입을 모른다. 비교 함수가 안다 — C 의 다형성

(x > y) - (x < y)

double 뺄셈은 int 로 잘리고, int 뺄셈은 오버플로. 이 형태가 모든 타입에 안전

b - a

인자 순서를 바꾸면 내림차순

콜백 — "이 일이 생기면 이 함수를 불러라"

7교시

```
typedef void (*Callback)(double);  
static Callback on_alert = NULL;           // 등록 슬롯  
  
void register_alert(Callback cb) { on_alert = cb; }  
void feed(double t) {                     // 라이브러리 쪽  
    if (t > 35.0 && on_alert) on_alert(t); // NULL 검사 후 호출  
}  
  
void beep(double t) { printf("BEEP! %.1f\n", t); }  
  
feed(36.0);           // 아무 일 없음  
register_alert(beep);  
feed(36.0);           // BEEP! 36.0
```



feed 를 짰 사람은 beep 이 존재하는지 모른다. 다음 주 attachInterrupt(pin, handler), 타이머, 버튼 — ESP32 코드의 절반이 콜백 등록.

함수 포인터도 포인터 — NULL 검사 없이 부르면 세그폴트 (Day 2 1교시).

상태 머신 — enum 인덱스 = 핸들러 테이블

7교시

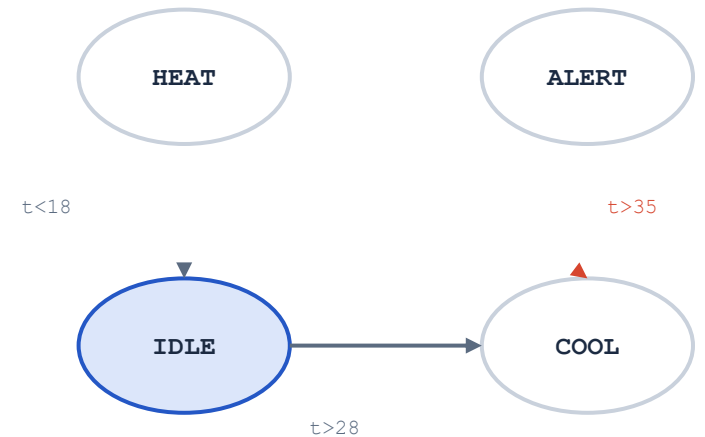
TABLE[st](t): switch 없이 상태가 곧 인덱스

```
typedef enum { ST_IDLE, ST_HEAT, ST_COOL, ST_ALERT, ST_COUNT } State;

static State on_idle(double t) { GPIO &= ~ALL;
    if (t < 18) return ST_HEAT; if (t > 28) return ST_COOL; return ST_IDLE; }
static State on_heat(double t) { GPIO |= HEATER;    if (t >= 20) return ST_IDLE; return
ST_HEAT; }
static State on_cool(double t) { GPIO |= FAN;        if (t > 35) return ST_ALERT; if (t <= 26)
return ST_IDLE; return ST_COOL; }
static State on_alert(double t) { GPIO |= ALARM|FAN; if (t <= 30) return ST_COOL; return
ST_ALERT; }

typedef State (*Handler)(double);
static const Handler TABLE[ST_COUNT] = { [ST_IDLE]=on_idle, [ST_HEAT]=on_heat,
                                           [ST_COOL]=on_cool, [ST_ALERT]=on_alert };

State next = TABLE[st](temps[i]); // 핵심 한 줄
```



```
T=17.0 IDLE -> HEAT  GPIO=0x0
T=19.0 HEAT  -> HEAT  GPIO=0x1
T=36.0 COOL  -> ALERT GPIO=0x2
T=31.0 ALERT -> ALERT GPIO=0x6
```

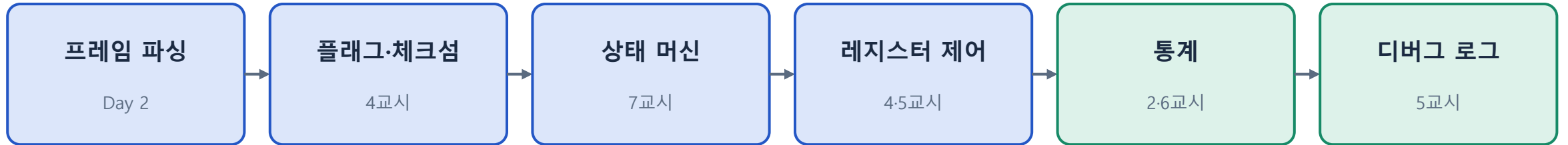
△ TABLE = { on_idle, on_cool, on_heat, on_alert } — 컴파일·실행 모두 되고, HEAT 에서 팬이 돈다. 지정 초기화로 enum 과 묶어라. 오늘 함정은 조용히 틀린다.

📺 D3-7 상태 머신 시뮬레이터 — 시나리오 재생

종합 실습 — 온도 조절기 컨트롤러

8교시

파일 5개 (main.c · frame.c/h · fsm.c/h · gpio.h · Makefile) · 경고 0 · -DDEBUG 로 전이 로그



```
$ make && ./controller < temps.txt
IDLE 3
HEAT 2
COOL 3
ALERT 2
top3: 36.0 33.0 31.0
bad=0 final GPIO=0x6
$ make CFLAGS="-Wall -Wextra -g -DDEBUG" && ./controller < temps.txt
T=25.0 IDLE -> IDLE GPIO=0x0 ...
```

채울 곳 셋: cmp_temp_desc (6교시), realloc 블록 (Day 2),
qsort + top3 출력.
조건: TABLE 은 지정 초기화 · 핀 제어는 GPIO_SET/CLR 매크로
로만 · visits[ST_COUNT] 로 상태별 횟수.
미완성이어도 fsm_step + 테이블 + visits 가 있으면 통과.

제출 체크리스트

8교시



make clean && make

경고 0개 · CFLAGS 로 -DDEBUG 빌드도 성공



[ST_HEAT] = on_heat

TABLE 이 지정 초기화로 되어 있음



GPIO_SET / GPIO_CLR

핀 제어가 매크로로만 (직접 != 금지)



int visits[ST_COUNT]

상태별 체류 횟수를 enum 인덱스로



worksheet_day3.html

8교시까지 채점 → 결과 코드 · 오답 정리 캡처

```
# Makefile
CFLAGS = -Wall -Wextra -g
controller: main.o frame.o fsm.o
    $(CC) $(CFLAGS) -o $@ $^
main.o: main.c frame.h fsm.h gpio.h
fsm.o: fsm.c fsm.h gpio.h
```

제출 폴더: c_day3/controller/

오늘의 여섯 문장

1

구조체 대입은 통째 복사.
함수 인자는 `const T *` 가 기본

 D3-1

2

필드는 자기 크기의 배수 주소에. 큰 것부터 놓으면 패딩이 사라진다.
`union` 은 같은 자리를 나눠 쓴다

 D3-2

3

켜기 `|=`, 끄기 `&=`, 토글 `^=`, 검사 `>> n & 1`.
필드 쓰기는 지우고 넣는다. 비트 연산은 `unsigned` 로

 D3-3 · D3-4

4

매크로는 텍스트 치환 — 괄호 셋, 부수효과 금지.
`volatile` 은 "매번 진짜로 읽어라"

 D3-5

5

함수 이름은 주소.
`qsort` 는 비교 함수가 타입을 안다

 D3-6

6

콜백은 미리 등록,
상태 머신은 `enum` 인덱스 = 핸들러 테이블

 D3-7

Day 4 예고

파일 I/O · 디버깅 · 종합 프로젝트

오늘 만든 것들이 내일 이렇게 이어집니다

```
fopen("temps.txt", "r") · fgets · fwrite
```

파일 I/O — 표준 입력 대신 프로그램이 직접 파일을 연다

```
gdb ./controller → break fsm_step → print st
```

예제 7-3 의 조용한 버그를 실행을 따라가며 잡는다

링 버퍼 + 상태 머신 + 파일 로깅

종합 프로젝트 — 4일치를 한 프로그램으로

제출: c_day3/controller/ (파일 5개) + worksheet_day3.html 결과 · 내일 09:00